WISEConv: A Worklist-Driven Masked-Convolution Runtime for GPUs

Anonymous Author(s)

Abstract

Masked convolution promises to cut the per-frame inference latency of vision models by computing only positions an upstream policy marks active, yet existing paths sacrifice either throughput or useful-compute ratio and fail to convert sparsity into proportional latency reduction. We present WISEConv, a masked-convolution runtime that derives an active-output worklist from the upstream mask and uses it to drive the dense tiled pipeline, securing both a high useful-compute ratio and high throughput by restricting computation to active positions while retaining the regular dense datapath. To amortize construction overhead and keep the compute pipeline fully utilized despite per-frame-varying worklist lengths, WISEConv co-designs construction and consumption: compatible operators reuse the constructed worklist without reconstruction, the worklist emission order preserves tile-local spatial reuse, and a data-driven adaptive decomposition adjusts parallel work to the activity level without host synchronization. Across four perception models and six GPU platforms (four discrete, two embedded), WISEConv reduces full-model latency by 22.1–46.5% relative to the fastest competing path and per-frame energy by 28.0–59.6% on the embedded platforms.

Keywords: Sparse convolution, dynamic spatial sparsity, GPU runtime, deep learning systems, efficient inference

1 Introduction

Dense convolution dominates the per-frame inference latency of many latency-critical vision tasks [2, 9, 37], yet on a given input much of that computation is spatially redundant. A common remedy is masked convolution: an upstream policy marks active positions, and the operator computes only those. The mask may derive from event-camera increments [7, 22], temporal differences [16, 27], or learned spatial gates [20, 34]. Regardless of source, it poses one operator-level problem: compute the convolution only at the active positions of the dense feature grid.

Dense GPU convolution obtains high throughput from tiled, coordinate-driven execution. Each tile enumerates a contiguous block of output coordinates; those coordinates fix each output position's receptive field and write location, while the reduction over channels and kernel offsets follows a regular datapath. Neighboring coordinates share overlapping receptive fields, yielding predictable memory access and data reuse. However, unlike static sparsity in matrix multiplication, masked convolution selects positions at runtime per frame, requiring the operator to access scattered, irregularly-overlapping neighborhoods and handle a varying workload, conflicting with tiled execution.

Existing masked-convolution systems preserve this regularity or eliminate wasted work, but not both. *Tile skipping* [7, 16, 27, 29] retains the dense pipeline and suppresses a tile only when all its outputs are inactive, so a single active position triggers computation for the entire tile. *Gather-scatter* [20, 34, 38] (as an execution path) instead extracts active positions exactly, materializes receptive fields, and scatters the computation results to the dense grid, sacrificing regularity and incurring extraction, irregular access, and writeback overhead.

We characterize this trade-off along two axes (§2.2): *throughput* (operations per second) and *useful-compute ratio* (the fraction of issued operations that produce needed outputs). In our sparsest workload (the DynConv-gated model, §6.2), the masks render 84.4% of the convolution work unnecessary, yet neither approach turns this into proportional latency reduction. Tile skipping [27] suffers a useful-compute ratio of only 49.1% and cuts latency by at most 40.6%. Gather-scatter drives the useful-compute ratio near one but sustains only 9.3% of tile skipping's throughput, running 3.1× slower than the dense path.

Our key insight is that position-level selectivity does not require abandoning the dense convolution pipeline. A dense tile already derives its reads and writes from output coordinates, so replacing its contiguous coordinate block with an active-output worklist changes which outputs are computed without changing the per-position reduction, the dense tensor layout, or the weight layout. With this substitution, the operator runs as a dense tiled pipeline, raising the useful-compute ratio toward one without materializing input patches.

Translating this into latency reduction requires handling two challenges. First, *construction efficiency*: construction scans the mask, so its cost scales with the layer's spatial grid, not with the active count or channel widths, while the compute it enables shrinks with both. Construction's share of latency therefore grows as activity falls and channels thin, and accumulates across operators when every layer rebuilds. Second, *end-to-end worklist efficiency*: the worklist's order and length both affect its consumption throughput. An unordered worklist scatters the input neighborhoods that adjacent work touches, lowering locality; its length falls below the dense grid and shifts with layer and input, so no fixed tiling keeps the GPU full, and because the length is device-resident, any host-side decision that depends on it costs a synchronization on the per-frame path. Both effects sharpen as activity falls.

We present WISEConv (**W**orklist-dr**I**ven ma**S**k**E**d **Conv**olution), a masked-convolution runtime that fuses position-level selectivity into the dense tiled pipeline, securing both a high useful-compute ratio and high throughput by co-designing worklist construction and consumption. Construction is cheap and infrequent: a single mask-space pass fuses output-mask propagation with per-tile compaction, operating on only mask and coordinate metadata; compatible operators reuse valid metadata instead of reconstructing it. Consumption is dense-like: construction emits contiguous per-tile segments ordered by the dense tile traversal, giving the compute stage tile-local structure without global sorting. Compute then decomposes parallel work adaptively according to device, layer shape, and activity, choosing each operator's configuration from offline calibration. For temporally correlated inputs, it tracks activity across frames and sustains GPU utilization without host synchronization.

This paper makes the following contributions:

- We characterize masked-convolution latency along two axes, useful-compute ratio and throughput, exposing why existing work does not translate upstream sparsity into proportional latency reduction.
- We design WISEConv, which fuses position-level selectivity into the dense tiled convolution pipeline and co-designs worklist construction and consumption to secure both axes jointly.
- We evaluate WISEConv on four perception models across four discrete GPUs and two Jetson platforms. WISEConv reduces full-model latency relative to the fastest baseline by 34.2–46.5% on the discrete GPUs and 22.1–38.6% on the Jetson platforms, and cuts per-frame energy over dense convolution by 28.0–59.6% on the Jetson platforms.

2 Background and Motivation

2.1 Masked Convolution

Convolution runs inside the perception-action loop of many vision applications [2, 31, 35, 37, 41], where per-frame inference latency bounds system responsiveness [8–10, 17]. To cut this latency, masked convolution exploits dynamic spatial sparsity: an upstream policy emits a spatial mask, and the operator computes only the marked positions. The mask comes from sources including event-camera increments [7, 22], temporal residuals between video frames [3, 16, 26, 27, 36], or learned input-dependent gates [12, 20, 34, 38]. The event and temporal policies mark positions that changed since the previous frame; the learned gates mark positions the task treats as relevant. These sources differ in sensing modality and policy, but all expose the same dynamic spatial sparsity on a dense 2D feature grid, so a single masked-convolution runtime serves them all.

Masked convolution keeps the tensors and arithmetic of dense convolution and restricts only which output coordinates it computes. It operates on an input feature tensor $\mathbf{x} \in \mathbb{R}^{H_{in} \times W_{in} \times C_{in}}$, a weight tensor $\mathbf{w} \in \mathbb{R}^{k \times k \times C_{in} \times C_{out}}$ of kernel size k, and an output $\mathbf{y} \in \mathbb{R}^{H_{out} \times W_{out} \times C_{out}}$ on a coordinate grid $\mathcal{G} = \{1, \ldots, H_{out} W_{out}\}$, where each coordinate $p \in \mathcal{G}$ indexes a spatial position (h, w) and has a $k \times k$ receptive field $\mathcal{N}(p)$ in $\mathbf{x}$. An upstream policy marks the coordinates to compute in an output mask $M_{out} \in \{0,1\}^{H_{out} \times W_{out}}$. Spatial gating predicts M_{out} directly; event and temporal policies instead supply an input mask $M_{in} \in \{0,1\}^{H_{in} \times W_{in}}$ whose active positions propagate their influence along the receptive field, so p stays active whenever any input in $\mathcal{N}(p)$ is active,

$$M_{out}[p] = \begin{cases} 1, & \exists\, r \in \mathcal{N}(p) \text{ s.t. } M_{in}[r] = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

M_{out} thus identifies the output positions that the execution path is required to compute, each by aggregating $\mathbf{x}$ over $\mathcal{N}(p)$ with the weights $\mathbf{w}$.

An execution path issues computation over a sequence of position slots C, where each slot is drawn from $\mathcal{G} \cup \{\perp\}$; $\perp$ denotes a padding slot that carries no output coordinate. The path must satisfy the *coverage contract*: every active coordinate is included, $\{p : M_{out}[p] = 1\} \subseteq \{c \in C : c \neq \perp\}$. For each non-$\perp$ slot $p \in C$ it computes

$$\mathbf{y}[p] = \sum_{i=1}^{k^2} \mathbf{w}_i \, \mathbf{x}[\mathcal{N}_i(p)], \tag{2}$$

where $\mathcal{N}_i(p)$ is the i-th position in $\mathcal{N}(p)$ and $\mathbf{w}_i \in \mathbb{R}^{C_{in} \times C_{out}}$ the corresponding weight slice. Positions the path does not compute retain their existing value in $\mathbf{y}$: masked convolution updates the output in place, so $\mathbf{y}[p]$ for $p \notin \{c \in C : c \neq \perp\}$ carries over from the previous frame (event and temporal policies) or from a one-time dense initialization (spatial gating). Coverage is therefore the correctness condition: every position the policy marks as changed is recomputed, and every other position is already correct by construction. Beyond coverage, a path may issue inactive or $\perp$ slots; dense convolution is the extreme where $\{c \in C : c \neq \perp\} = \mathcal{G}$. The policy fixes the accuracy and the required work, while C is determined by the path.

2.2 The Sparsity-to-Latency Gap

Existing systems execute a mask along one of two paths. *Tile skipping* [7, 16, 21, 26, 29, 36] inherits the dense pipeline's tile-based computation, skipping a tile only when all its coordinates are inactive and otherwise computing the whole tile; DeltaCNN [27] fuses a selective path into the kernel, taken when a tile is estimated to be mostly empty, to cut wasted work. *Gather-scatter* [20, 38] instead extracts exactly the active coordinates, computes, and scatters them back, removing the inactive arithmetic but adding extraction, irregular access, and writeback overhead; DynConv [34] adopts

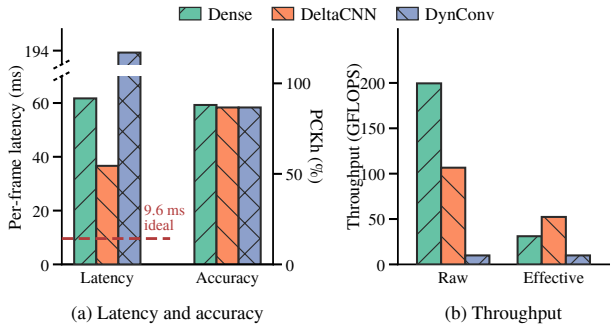

Figure 1. Statistics of DynConv [34], a pose estimator with learned spatial gating, on the MPII human-pose dataset [1] and Jetson AGX Orin, comparing the native cuDNN path (Dense) against the two sparse paths. (a) End-to-end average per-frame latency (left axis; the dashed line marks the dense latency scaled by the active ratio aggregated across layers) and PCKh accuracy (right axis; PCKh evaluates the percentage of keypoints localized within half the head size). The sparse paths apply the same mask, so their accuracy coincides. (b) Raw throughput T and effective throughput T_{eff}, aggregated across the model's convolution layers. The dense useful-compute ratio equals the active ratio set by DynConv's spatial gating ($\alpha \approx 15.6\%$).

a model structure suited to gather-scatter, where one gather feeds several consecutive layers before a single scatter, amortizing that overhead.

These paths fail for different reasons, so we separate policy sparsity from execution efficiency. The policy fixes its active ratio α, the fraction of output-grid positions marked active. An execution path instead determines its useful-compute ratio η and throughput T, which combine into effective throughput T_{eff}:

$$\alpha = \frac{\|M_{out}\|_0}{|\mathcal{G}|}, \qquad \eta = \frac{\|M_{out}\|_0}{|C|} \le 1, \qquad T = \frac{2|C|k^2 C_{in} C_{out}}{t}, \quad T_{\text{eff}} = \eta T. \tag{3}$$

For $|C| > 0$, η captures inactive and padding slots in the issued schedule, while T captures how quickly the path processes those slots. The latency t includes all auxiliary mask and coordinate work, so such work lowers T without changing η; the factor of two counts each multiply-accumulate as two FLOPs. Consequently, T_{eff} is the rate at which the path computes required outputs and directly determines their latency.

Figure 1 traces this on Jetson AGX Orin for the sparsest workload in our evaluation. The policy marks 84.4% of the convolution work unnecessary, and the sparse paths discard it with negligible accuracy loss, so the remaining work sets a 9.6 ms latency floor (Fig. 1a, dashed). Neither path approaches it: tile skipping (DeltaCNN [27]) reaches 36.6 ms, still 3.8× the floor, and gather-scatter, even with DynConv built to amortize its overhead, runs at 193.5 ms, far slower than dense convolution itself. The gap traces to low effective throughput on both paths. Tile skipping holds throughput near half of dense but at a useful-compute ratio of only 49.1%; gather-scatter lifts the useful-compute ratio near one, yet extraction and data movement collapse its throughput to 5.0% of dense (Fig. 1b). Neither path efficiently converts the GPU's capability into effective throughput for masked convolution.

2.3 Distinction from Related Sparse Execution

Related sparse-execution work differs from WISEConv's masked convolution in representation, operator, sparsity type, or platform. 3D sparse convolution (SpConv v2 [30], TorchSparse++ [32, 33], Minuet [39], MinkowskiEngine [6], submanifold sparse convolution [15]) is built to organize computation over an unordered list of active voxels, constructing coordinate maps that route irregular input coordinates to outputs; its input is sparse by construction rather than a dense grid under a runtime mask, so it is related infrastructure rather than a baseline. Sparse matrix multiplication (DTC-SpMM [11], MaxK-GNN [28], Sputnik [13]) reorganizes a sparse matrix operand so the dense datapath can consume it; its sparsity lives in the matrix structure of a different operator, not in a spatial mask over a convolution grid. Structured weight sparsity (2:4 tensor cores [24]) and sparse-format compilers (TACO [19], SparseTIR [40]) exploit a static structural sparsity fixed at compile time in the weights or a known format, rather than a mask discovered at runtime over the spatial grid. DPACS [14] resolves similar dynamic spatial sparsity in CNN feature maps, but on a custom accelerator instead of a commodity GPU.

3 WISEConv Overview

Dense tiled convolution already consumes a stream of output coordinates: each coordinate determines the receptive-field reads and output write, while its reduction over channels and kernel offsets remains regular. WISEConv exploits this interface by replacing the dense coordinate sequence with a worklist of requested output positions, thereby introducing position-level selectivity into the tiled pipeline. Removing unrequested positions from the schedule raises the useful-compute ratio, but issuing fewer operations alone does not guarantee lower latency. To translate selectivity into latency reduction, the worklist must be inexpensive to construct and must be consumed with both a high useful-compute ratio and high throughput. WISEConv therefore co-designs a construction stage and a compute stage around these interdependent requirements, as Figure 2 summarizes.

Construction stage. To prevent worklist construction from erasing the savings of selective compute, WISEConv confines construction to mask and coordinate metadata, never reading feature or weight tensors. In one mask-space

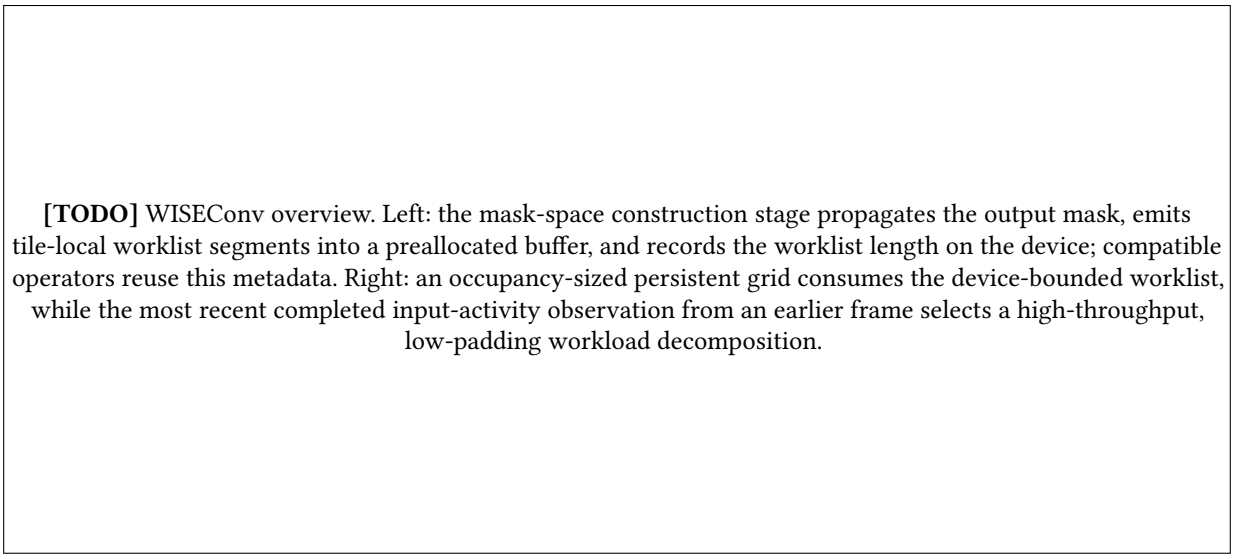

Figure 2. The WISEConv two-stage pipeline. Construction operates only on mask and coordinate metadata, emits tile-locally ordered worklist segments, and reuses the resulting metadata across compatible operators. Compute bounds persistent work on the device and uses a completed, temporally lagged input-activity observation to adapt the workload decomposition without host synchronization.

pass, it fuses the output-mask propagation in Eq. 1 with tile-local compaction, producing M_{out} and a worklist $\mathcal{W}$; the requested coordinates from each spatial tile form a segment in local dense-traversal order, preserving spatial grouping without global ordering. WISEConv stores $\mathcal{W}$ in a buffer preallocated for $|\mathcal{G}| = H_{out}W_{out}$ coordinates, the static upper bound on $|\mathcal{W}|$. A device-resident counter records $n_{\mathcal{W}} = |\mathcal{W}|$, so the host neither reads the active count nor allocates storage at runtime.

Even this metadata-only pass must scan the layer's mask grid. WISEConv therefore propagates and reuses the resulting mask and worklist metadata across compatible operators instead of reconstructing it, amortizing one construction across the compatible operator sequence. The fused pass lowers the cost of each construction, while cross-operator reuse reduces how often it occurs; together, they address Construction Efficiency and Amortization.

Compute stage. The worklist length varies across layers and inputs and remains device-resident; synchronizing its current value to the host would stall the per-frame path, yet the workload decomposition must adapt because its padded work and exposed parallelism govern useful-compute ratio and GPU throughput. To execute without a host-visible bound, WISEConv launches only enough persistent thread blocks to fill the GPU's concurrent block residency and lets them consume $\mathcal{W}$ through a grid-stride loop bounded by device-resident $n_{\mathcal{W}}$, so the launch footprint tracks resident capacity rather than the full dense output grid.

To choose a decomposition without querying the exact $n_{\mathcal{W}}$, WISEConv builds from representative temporal traces a table that maps input-activity levels to configurations that sustain throughput while limiting padding, then indexes it with the most recent completed observation from one or more frames earlier; temporal correlation keeps this lagged signal informative, while completed-only polling prevents selection from blocking. The device count thus bounds the current work, while the data-driven signal chooses its decomposition; together, they sustain useful-compute ratio and GPU throughput as the worklist length varies without synchronizing the per-frame path.

Neither stage suffices alone: excessive construction cost, a low useful-compute ratio, or low consumption throughput can each erase the benefit of issuing less convolution work. By controlling all three jointly, WISEConv converts upstream spatial sparsity into end-to-end latency reduction.

4 WISEConv Design

The design follows the worklist from its construction and propagation across compatible operators (§4.1) to its consumption by the convolution pipeline (§4.2).

4.1 Worklist Construction and Reuse

4.1.1 Tile-Local Worklist Construction. Constructing a tight schedule already requires a scan of the output grid, but the resulting coordinates must also retain enough spatial structure for efficient consumption. A globally stable compaction preserves the complete dense traversal order, but obtaining its device-wide offsets requires a prefix scan or an additional pass. Independent appends avoid that coordination, but scatter nearby positions across the schedule.

Materializing active receptive fields, as gather-scatter does, regularizes the compute input at the cost of feature traffic proportional to the active count and k^2C_{in}. The useful middle ground is therefore a metadata-only schedule that preserves order locally rather than globally.

WISEConv realizes this middle ground with contiguous, tile-local worklist segments. Because the number of active positions is not known before the pass, it preallocates a coordinate buffer with capacity $|\mathcal{G}| = H_{out}W_{out}$, the static upper bound on $|\mathcal{W}|$. Algorithm 1 details the resulting single pass. Line 1 initializes the device-resident length counter. Lines 2–7 partition the output grid into $B_\tau \times B_\tau$ construction tiles, derive the corresponding entries of M_{out} using Eq. 1, and compact each tile's active coordinates into a local sequence $\mathcal{P}_\tau$ in dense-traversal order. When a learned gate supplies M_{out} directly [20, 34], Lines 4–5 are omitted and Line 7 compacts the supplied mask. Lines 8–13 count the compacted coordinates, atomically reserve $[\text{base}, \text{base} + n_\tau)$ from the counter, and copy $\mathcal{P}_\tau$ into that interval. After all tiles complete, the reserved intervals collectively store $\mathcal{W}$, and the counter records $n_{\mathcal{W}}$. The layer shape statically determines both the buffer capacity and the construction launch geometry, while only the device observes the data-dependent worklist length. Construction therefore needs neither runtime allocation nor a host-visible active count.

To relate the worklist to the useful-compute ratio of Eq. 3, we insert its length $n_{\mathcal{W}}$ between the required-position count and the issued-slot count. For $n_{\mathcal{W}} > 0$,

$$\eta = \underbrace{\frac{\|M_{out}\|_0}{n_{\mathcal{W}}}}_{\eta_{\text{construct}}} \cdot \underbrace{\frac{n_{\mathcal{W}}}{|C|}}_{\eta_{\text{compute}}}. \tag{4}$$

The first factor, $\eta_{\text{construct}}$, compares required positions with worklist entries; the second, η_{compute}, compares worklist entries with issued slots. We analyze $\eta_{\text{construct}}$ here and defer η_{compute} until the compute organization is defined in §4.2. The factorization decomposes η only; all stage costs remain reflected in throughput T.

Fresh construction introduces no construction-side expansion. Its tiles partition the output grid, and their disjoint reservations place each active coordinate exactly once in $\mathcal{W}$. Thus $n_{\mathcal{W}} = \|M_{out}\|_0$ and $\eta_{\text{construct}} = 1$ for a nonempty worklist. Because coverage constrains membership rather than order, the order in which tiles reserve their intervals does not affect correctness. The within-interval traversal order is retained for locality.

WISEConv deliberately forgoes a global worklist order, allowing construction tiles to reserve contiguous segments independently without device-wide ordering coordination. Within each segment, active coordinates follow the tile's dense traversal, giving compute a spatially grouped coordinate run but offering no locality guarantee across segment boundaries. As shown in §6.3, tile-local ordering achieves compute-stage throughput comparable to that under global ordering; avoiding the additional ordering pass is therefore the better end-to-end trade-off.

Algorithm 1 Tile-Local Worklist Construction

Require: Input mask M_{in}, output grid $\mathcal{G}$, tile size B_τ
Ensure: Mask M_{out}, worklist $\mathcal{W}$, length $n_{\mathcal{W}} = |\mathcal{W}|$

1: $n_{\mathcal{W}} \leftarrow 0$
2: **for** each $B_\tau \times B_\tau$ spatial tile τ **in parallel do**
3: **for** each position $p \in \tau$ **in parallel do**
4: active$(p) \leftarrow \exists\, r \in \mathcal{N}(p) : M_{in}[r] = 1$
5: $M_{out}[p] \leftarrow$ active(p)
6: **end for**
7: $\mathcal{P}_\tau \leftarrow$ Compact(τ, M_{out})
8: $n_\tau \leftarrow |\mathcal{P}_\tau|$
9: ▷ *Atomically reserve n_τ worklist entries.*
10: base $\leftarrow n_{\mathcal{W}}$
11: $n_{\mathcal{W}} \leftarrow n_{\mathcal{W}} + n_\tau$
12: **for** each $0 \le j < n_\tau$ **in parallel do**
13: $\mathcal{W}[\text{base} + j] \leftarrow \mathcal{P}_\tau[j]$
14: **end for**
15: **end for**

4.1.2 Cross-Operator Worklist Reuse.

Construction has a channel-independent grid-scan cost. Each pass examines all $H_{out}W_{out}$ positions and, when deriving M_{out} from M_{in}, performs $O(k^2)$ mask work per position, whereas required convolution work scales as $O(\alpha H_{out}W_{out}k^2C_{in}C_{out})$. Repeating this scan across consecutive same-grid layers that introduce no new required positions is therefore redundant. WISEConv instead reuses one constructed worklist across each compatible layer sequence, amortizing its construction cost over the sequence.

For layer l, let $\mathcal{G}^l$ denote its output grid, M^l_{out} its output mask, $\mathcal{W}^l$ its worklist, and $n^l_{\mathcal{W}}$ its device-resident length. WISEConv propagates the mask and worklist metadata alongside the feature tensor. From layer semantics, WISEConv statically infers that consecutive layers are reuse-compatible when $\mathcal{G}^{l+1} = \mathcal{G}^l$ and

$$\{p \in \mathcal{G}^{l+1} \mid M^{l+1}_{out}[p] = 1\} \subseteq \{p \in \mathcal{G}^l \mid M^l_{out}[p] = 1\}. \tag{5}$$

Because $\mathcal{W}^l$ already satisfies coverage for M^l_{out}, Eq. 5 guarantees coverage for M^{l+1}_{out} as well. WISEConv therefore sets $\mathcal{W}^{l+1} = \mathcal{W}^l$ and $n^{l+1}_{\mathcal{W}} = n^l_{\mathcal{W}}$ without reconstruction. WISEConv applies the same condition recursively to subsequent layers, forwarding the original $\mathcal{W}^l$ and $n^l_{\mathcal{W}}$ through the entire compatible sequence.

Equality in Eq. 5 covers same-grid 1×1 convolutions, mask-preserving activations, and inference-time normalization and leaves $\eta_{\text{construct}}$ unchanged. Strict inclusion results from mask-narrowing operators in the upstream policy, such as the activation-fused update truncation that DeltaCNN uses to increase downstream sparsity [27]. WISEConv therefore

retains $\mathcal{W}^{l+1}$ as a valid superset, trading the resulting lower $\eta_{\text{construct}}$ for one fewer construction pass without violating coverage.

A change in the output grid violates the geometry constraint, while receptive-field expansion may violate Eq. 5 by introducing positions outside the upstream mask; either case invokes the construction pass of §4.1.1 to emit a new tight worklist. A lower $\eta_{\text{construct}}$ alone does not trigger reconstruction because tightening a valid superset would pay another grid scan. Reuse avoids repeated construction inside each compatible sequence, but common receptive-field-expanding layers such as 3×3 convolutions create rebuild points throughout the network. The resulting device-resident worklist lengths vary across layers and frames, so compute still faces a dynamic problem size.

4.2 Variable-Length Worklist Execution

Construction and reuse fix $\eta_{\text{construct}}$ before compute begins; the implicit-GEMM decomposition controls the remaining η_{compute} and also affects throughput T. For a standard ungrouped batch-1 convolution, dense implicit GEMM uses $M = H_{out}W_{out}$ output-position rows, $N = C_{out}$ output-channel columns, and reduction depth $K = k^2C_{in}$. It decomposes the output into $B_M \times B_N$ tiles and traverses each tile's reduction in B_K-wide steps. WISEConv preserves the N and K dimensions and this tiled pipeline, but changes the M-dimension coordinate source: instead of enumerating the dense output grid, it reads the $n_{\mathcal{W}}$ coordinates in $\mathcal{W}$. Compute therefore groups $\mathcal{W}$ into B_M-coordinate tiles, padding only the final tile. For $n_{\mathcal{W}} > 0$, this issues $|C| = B_M\lceil n_{\mathcal{W}}/B_M\rceil$ position slots, giving

$$\eta_{\text{compute}} = \frac{n_{\mathcal{W}}}{B_M\,\lceil n_{\mathcal{W}}/B_M\rceil}. \tag{6}$$

The final tile contains at most $B_M - 1$ padding slots, making this overhead most pronounced for a short worklist. The same B_M also controls data reuse, block efficiency, and exposed parallelism, so maximizing η_{compute} need not maximize throughput T.

For dense convolution, (M, N, K) follow from the layer shape, allowing the backend to select (B_M, B_N, B_K) before execution. In the worklist path, the M-dimension extent is $n_{\mathcal{W}}$, which varies across invocations and remains unknown to the host. This creates two problems: executing the current worklist without exposing its length to the host, and selecting a decomposition suited to that length. §4.2.1 addresses the former with persistent execution, while §4.2.2 addresses the latter through trace-calibrated selection.

4.2.1 Persistent Worklist-Driven Convolution. Under the coordinate-source substitution above, dense row m uses the m-th coordinate p_m of the output-grid traversal, whereas the worklist path reads $\mathcal{W}[m]$. Because each coordinate determines its receptive-field and output addresses, this substitution changes only the input gathers and output stores;

Algorithm 2 Persistent Worklist-Driven Implicit GEMM

Require: Device-resident $(\mathcal{W}, n_{\mathcal{W}})$, persistent-grid capacity g_{cap}, tensors $(\mathbf{x}, \mathbf{w})$, and tile shape (B_M, B_N, B_K)
Ensure: Output $\mathbf{y}$ (updated at worklist coordinates)

1: $n_{\text{work}} \leftarrow \lceil n_{\mathcal{W}}/B_M\rceil\lceil C_{out}/B_N\rceil$
2: **for** $t_i = 0, \ldots, g_{\text{cap}} - 1$ **in parallel do**
3: **while** $t_i < n_{\text{work}}$ **do**
4: $(b_m, b_n) \leftarrow \text{Unflatten}(t_i)$
5: $\mathcal{P} \leftarrow \mathcal{W}[b_mB_M : \min((b_m + 1)B_M, n_{\mathcal{W}})]$
6: Pad $\mathcal{P}$ to B_M entries with $\perp$
7: $c_1 \leftarrow b_nB_N, \quad c_2 \leftarrow (b_n + 1)B_N$
8: $\mathbf{Y} \leftarrow \mathbf{0}_{B_M \times B_N}$
9: **for** $j = 1, \ldots, k^2$ **do**
10: **for** $r = 0, \ldots, \lceil C_{in}/B_K\rceil - 1$ **do**
11: $r_1 \leftarrow rB_K, \quad r_2 \leftarrow (r + 1)B_K$
12: $\mathbf{X} \leftarrow [\,\mathbf{x}[\mathcal{N}_j(p), r_1 : r_2]\,]_{p\in\mathcal{P}}$, zero-padding $\perp$
13: $\mathbf{W} \leftarrow \mathbf{w}_j[r_1 : r_2, c_1 : c_2]$
14: $\mathbf{Y} \leftarrow \mathbf{Y} + \mathbf{X} \cdot \mathbf{W}$
15: **end for**
16: **end for**
17: $\mathbf{y}[\mathcal{P}, c_1 : c_2] \leftarrow \mathbf{Y}$, skipping $\perp$
18: $t_i \leftarrow t_i + g_{\text{cap}}$
19: **end while**
20: **end for**

feature and weight layouts and the regular B_K-tiled reduction remain unchanged, and no receptive-field patches are materialized. Reading contiguous ranges of $\mathcal{W}$ carries construction's tile-local grouping into the input gathers.

A conventional kernel launch requires the host to determine its thread-block count before execution, but $n_{\mathcal{W}}$ is available only on the device. Reading the count on the host would synchronize every operator, while launching for the output-grid upper bound would submit blocks in proportion to dense work. WISEConv instead derives each configuration's persistent-grid capacity g_{cap} from concurrent device residency and launches an occupancy-sized grid of exactly that many blocks, independently of $n_{\mathcal{W}}$.

The exact work assignment then occurs entirely on the device (Algorithm 2). Line 1 derives the number of position-channel work items from $n_{\mathcal{W}}$, and Lines 2–3 instantiate g_{cap} persistent blocks and use this count as their device-side loop bound. For each item, Lines 4–8 recover its (b_m, b_n) indices, load and pad the corresponding B_M-coordinate slice, select its B_N output channels, and initialize a fixed $B_M \times B_N$ accumulator. Lines 9–16 then perform the regular implicit-GEMM reduction: each kernel offset and B_K-wide channel slice forms $\mathbf{X}$ and $\mathbf{W}$ directly from the dense tensors (Lines 12–13) and accumulates $\mathbf{XW}$ (Line 14). Line 17 writes the nonpadding rows to their worklist coordinates, after which Line 18 advances the block by g_{cap}. For clarity, the pseudocode omits the boundary checks required when C_{in} or C_{out} is not tile-aligned. Thus, $n_{\mathcal{W}}$ controls only the device-side work bound

and final position padding; the multiply shape and tensor layouts remain fixed. Selecting the configuration that executes this bound efficiently remains the problem addressed next.

4.2.2 Trace-Calibrated Adaptive Decomposition.

Persistent scheduling resolves the execution bound, but not the decomposition choice: for a fixed layer, the fastest configuration changes with $n_{\mathcal{W}}$. Each configuration comprises a tile shape (B_M, B_N, B_K) and a split-K factor S_K, defined as the number of disjoint partitions of the $K = k^2 C_{in}$ reduction for each position-channel tile. Algorithm 2 presents the $S_K = 1$ path, which writes each completed reduction directly. For $S_K > 1$, the standard split-K path first zeros $\mathbf{y}$ at the scheduled coordinates, then computes the S_K partitions independently and atomically accumulates their partial sums into $\mathbf{y}$. For layer l, let $\mathcal{B}_M^l$, $\mathcal{B}_N^l$, $\mathcal{B}_K^l$, and $\mathcal{S}_K^l$ denote the values supported for these four parameters, respectively, on the target GPU. Their Cartesian product

$$\mathcal{Q}_l = \mathcal{B}_M^l \times \mathcal{B}_N^l \times \mathcal{B}_K^l \times \mathcal{S}_K^l$$

defines the worklist configuration domain. Each $q \in \mathcal{Q}_l$ selects one supported value for each parameter and thereby specifies a concrete decomposition.

The length-sensitive effects are clearest in B_M and S_K. B_M trades final-tile padding and exposed position parallelism against within-tile reuse. Increasing S_K exposes S_K times as many work items when a short worklist would otherwise underfill the GPU, at the cost of coordinate rereads, output zeroing, and atomic accumulation. Because all split-K partitions cover the same coordinates, split-K affects throughput T but leaves C and η_{compute} unchanged.

WISEConv builds a level-to-configuration table $\mathcal{L}$, with one row $\mathcal{L}_l$ per layer, by quantizing α into D levels, $d = \max\{1, \lceil D\alpha \rceil\}$. For each layer l and level d, calibration constructs a synthetic worklist containing $\lceil d|\mathcal{G}^l|/D \rceil$ active coordinates with construction's tile-local structure, then measures $\widehat{t}_l(q; d)$, the latency of every $q \in \mathcal{Q}_l$ at that level. It records

$$q_{l,d}^* = \arg\min_{q \in \mathcal{Q}_l} \widehat{t}_l(q;\, d). \tag{7}$$

Because all worklist candidates execute the same synthetic worklist, $q_{l,d}^*$ is the decomposition with the highest measured throughput for layer l at level d. WISEConv stores this winner as $\mathcal{L}_l[d] = q_{l,d}^*$. When full-grid execution is semantics-compatible, calibration also compares a dense candidate as a special case and stores it in the same table if it is faster.

Although $\mathcal{L}_l$ makes configuration selection a lookup once d is known, obtaining each layer's current level directly would require transferring its device-resident activity count to the host. Waiting for each transfer would insert a synchronization at every layer. Making these transfers asynchronous and using their latest completed values would remove the waits, but not their cost: the runtime would enqueue and track one scalar transfer per layer per frame, making observation overhead scale with the layer count. WISEConv deliberately avoids both alternatives. It enqueues one asynchronous transfer of network-input activity per frame and consumes only the latest completed observation, necessarily from one or more frames earlier. The same lagged observation predicts all layers' levels, so one nonblocking input observation replaces per-layer transfers. For event and temporal workloads, this observation is predictive because layer masks derive from a shared network-input mask through receptive-field propagation and successive input masks correlate in time.

To fit this predictor, WISEConv profiles a set $\mathcal{F}$ of representative trace frames across the set $\mathcal{I}$ of layers that use it. It records $\{\alpha_0^f\}_{f \in \mathcal{F}}$ and $\{\alpha_l^f\}_{(f,l) \in \mathcal{F} \times \mathcal{I}}$, then partitions the observed input range $[\alpha_{\text{lo}}, \alpha_{\text{hi}}]$ into B equal-width buckets, where $\alpha_{\text{lo}} = \min_{f \in \mathcal{F}} \alpha_0^f$ and $\alpha_{\text{hi}} = \max_{f \in \mathcal{F}} \alpha_0^f$. Frame f receives index $b^f \in \{1, \ldots, B\}$:

$$b^f = \max\left\{1, \left\lceil \frac{B(\alpha_0^f - \alpha_{\text{lo}})}{\alpha_{\text{hi}} - \alpha_{\text{lo}}} \right\rceil\right\}. \tag{8}$$

Equal-width buckets preserve resolution in rare activity regimes because their ranges do not depend on sample frequency.

Given these frame groups, WISEConv fits a bucket-to-level prediction table $\mathcal{A}$, with one row $\mathcal{A}_l$ per layer. Separate rows are necessary because grid resolution, receptive-field size, and propagation depth transform the same input activity differently at each layer. For every bucket b, WISEConv computes the median layer activity $\widetilde{\alpha}_{l,b} = \text{median}_{f \in \mathcal{F}:\, b^f = b}\, \alpha_l^f$ over the frames assigned to b. It then quantizes this representative activity into the predicted target level for layer l in bucket b:

$$\hat{d}_{l,b} = \max\left\{1, \left\lceil D\widetilde{\alpha}_{l,b} \right\rceil\right\}. \tag{9}$$

The median minimizes absolute deviation within the bucket and is robust to outliers. WISEConv stores $\mathcal{A}_l[b] = \hat{d}_{l,b}$; an empty bucket copies the entry of its nearest populated neighbor.

At runtime, WISEConv composes $\mathcal{A}_l$ and $\mathcal{L}_l$ to translate the latest completed network-input activity observation into configurations for all layers. At frame f, let $f' < f$ denote the source frame of that input active-ratio observation. WISEConv clamps $\alpha_0^{f'}$ to $[\alpha_{\text{lo}}, \alpha_{\text{hi}}]$ and substitutes the result for α_0^f in Eq. 8 to obtain $\hat{b}^f$, avoiding extrapolation beyond trace coverage. If no observation has completed, it retains $\hat{b}^{f-1}$. It then selects

$$q_l^f = \mathcal{L}_l\left[\mathcal{A}_l[\hat{b}^f]\right]. \tag{10}$$

The observation may lag the current input, but it determines only the configuration; persistent compute still reads the current $n_{\mathcal{W}}^l$ on the device as the exact execution bound.

Prediction error can therefore affect performance but cannot omit scheduled coordinates.

Algorithm 3 Trace-Calibrated Nonblocking Configuration Selection

Require: Calibration frames $\mathcal{F}$, inference sequence $\mathcal{R}$, layers $\mathcal{I}$, D, and B
Ensure: Inference results for $\mathcal{R}$

1: **Offline:**
2: $\mathcal{L} \leftarrow$ level-to-configuration table from Eq. 7
3: $\{\alpha_0^f\}_f, \{\alpha_l^f\}_{f,l} \leftarrow$ input and layer active-ratio traces
4: $(\alpha_{\text{lo}}, \alpha_{\text{hi}}) \leftarrow (\min_{f\in\mathcal{F}} \alpha_0^f, \max_{f\in\mathcal{F}} \alpha_0^f)$
5: $\{b^f\}_f \leftarrow$ input buckets of $\{\alpha_0^f\}_f$ by Eq. 8
6: $\mathcal{A} \leftarrow$ bucket-to-level table from Eq. 9
7: **Online:**
8: *Queue* $\leftarrow$ empty asynchronous queue
9: $\hat{b}^0 \leftarrow$ calibrated initial input bucket
10: **for** inference frame $f = 1, \ldots, |\mathcal{R}|$ **do**
11: $\quad\hat{b}^f \leftarrow \hat{b}^{f-1}$
12: $\quad$**if** *Queue* is not empty **then**
13: $\quad\quad\alpha_0^{f'} \leftarrow$ dequeueLatest(*Queue*) $\quad\triangleright\ f' < f$
14: $\quad\quad\alpha_0^{f'} \leftarrow \text{clamp}(\alpha_0^{f'}, \alpha_{\text{lo}}, \alpha_{\text{hi}})$
15: $\quad\quad\hat{b}^f \leftarrow$ input bucket of $\alpha_0^{f'}$ by Eq. 8
16: $\quad$**end if**
17: $\quad\{q_l^f\}_l \leftarrow \{\mathcal{L}_l[\mathcal{A}_l[\hat{b}^f]]\}_l$
18: $\quad$**async do**
19: $\quad\quad\alpha_0^f \leftarrow$ current input-mask active ratio (Eq. 3)
20: $\quad\quad$Enqueue α_0^f into *Queue*
21: $\quad$**end async**
22: $\quad$Run frame f with $\{q_l^f\}_l$ via Algorithms 1 and 2
23: **end for**

Algorithm 3 connects the offline tables to nonblocking selection over the inference sequence $\mathcal{R}$. Lines 1–6 construct $\mathcal{L}$ from per-level kernel calibration and fit $\mathcal{A}$ by grouping the input and per-layer active-ratio traces. Lines 7–9 initialize the asynchronous queue and the input bucket used before an observation completes. For each frame, Lines 10–17 retain the previous bucket unless a newer completed observation is available, clamp and bucket that observation, and compose $\mathcal{A}_l$ with $\mathcal{L}_l$ to select every layer's configuration. Lines 18–21 asynchronously derive and enqueue the current input active ratio for a later frame, while Line 22 executes the current frame with the configurations already selected. An enqueued value becomes visible only after its device-to-host transfer completes; the bounded queue returns the newest visible value, discards older ones, and drops a new observation when no slot is free. Neither enqueue nor dequeue waits, so selection never stalls construction or compute.

Learned-gating models lack the shared input mask required by this predictor: each gated block derives its own mask from intermediate features. The per-layer asynchronous alternative would therefore enqueue one activity transfer for every gated layer on every frame. WISEConv avoids these transfers by assigning each such layer a fixed level calibrated from its gate statistics. The trace-driven mode thus derives all per-layer decompositions from one nonblocking input observation, whereas the fixed-level mode requires no runtime activity transfer for selection. Neither mode inserts per-layer device-to-host transfers into the per-frame path.

5 Implementation

We implement WISEConv as a CUDA C++ extension for PyTorch. The current backend executes FP32 tensors in channels-last layout and provides the mask-aware operators required by the evaluated networks: convolution, transposed convolution, pooling, pointwise activation, addition, concatenation, and upsampling. A model-conversion pass replaces compatible PyTorch modules while preserving the graph topology, convolution parameters, and learned weights. The upstream application continues to generate masks.

The construction kernel assigns contiguous output positions to each warp. A thread tests one receptive field and terminates once it finds an active input. A warp ballot identifies active lanes, population counts give their local ranks, and one atomic operation reserves a contiguous worklist segment for the warp. Active lanes then write their coordinates in lane order. This realizes the tile-local ordering in Section 4.1 without a device-wide prefix scan or sorting pass. An identity 1×1 convolution has the same input and output coordinate space; when its input already carries compatible mask and worklist metadata, the operator reuses that worklist and skips construction.

The compute kernel uses a persistent, tiled implicit-GEMM organization. It keeps partial sums in registers and stages input and weight tiles through shared memory. Supported architectures use asynchronous global-to-shared copies, with a synchronous path for earlier GPUs. Split-K exposes additional parallel work when the active-coordinate dimension alone cannot occupy the device. Transposed convolution follows the same output-driven organization, but partitions output coordinates by stride phase so that each phase visits only its valid kernel offsets.

Efficient scheduling depends jointly on the GPU, layer shape, and activity. We therefore autotune the sparse tile shape and split-K factor and cache the choice by device, convolution parameters, activity level, and execution policy. For layers whose state-update semantics permit dense evaluation, the candidate set can also include a cuDNN path. A sparse-only policy isolates the worklist-driven kernel, and the cache key distinguishes the two policies and the requested determinism mode. Autotuning and one-time weight preparation complete before the timed sequence described in Section 6.1.

For fixed input values, weights, and mask, WISEConv evaluates Eq. (2) at every active output coordinate. Floating-point reordering may produce small numerical differences from cuDNN, so we verify active outputs with numerical tolerances rather than requiring bitwise identity. This check separates backend correctness from the task-quality effect of the fixed upstream mask policy.

6 Evaluation

6.1 Experimental Setup

We evaluate WISEConv across four perception models, three mask sources, four GPU platforms, and three competing execution paradigms. The evaluation asks four questions: (1) does WISEConv reduce full-model latency relative to the fastest available path, (2) how effectively does it convert required-work reduction into latency reduction, (3) what overhead does worklist construction add, and (4) does replacing the masked-convolution backend change task output or accuracy?

6.1.1 Platforms. We evaluate four representative GPU platforms: two embedded SoCs (Jetson Xavier NX and Jetson AGX Orin), a laptop-class mobile GPU (RTX 4070 Laptop), and a desktop GPU (RTX 3080). These devices cover the hardware classes commonly used for onboard deployment and robotics development. The two Jetson systems anchor our target deployment setting, while the discrete GPUs test whether WISEConv generalizes beyond embedded SoCs.

6.1.2 Models, Tasks, and Mask Sources. Our selected workloads span three robotic perception tasks:

- **Event-based optical flow.** We evaluate FireFlowNet [25], a lightweight event-based optical-flow model that delivers high inference rates while retaining competitive accuracy, on MVSEC [42]. MVSEC contains event-camera sequences and corresponding ground-truth flow from indoor quadrotor flight and outdoor driving. Following Ev-Conv [7], consecutive 50-ms windows advance by 1 ms; events entering or leaving the window form the sparse increment from which we derive layer masks. We calibrate the sparsification policy on `indoor_flying1` and evaluate on the other three sequences.
- **Video object detection.** We evaluate YOLOv8n and YOLOv8m [18] on MOT16 [23], a collection of crowded pedestrian videos with annotated person boxes. Both models produce bounding boxes and confidence scores, while their different scales expose how model size affects sparse-execution efficiency. DeltaCNN [27] supplies the temporal mask mechanism by propagating thresholded activation deltas between successive frames.
- **Human pose estimation.** We use the DynConv pose model [34] on MPII [1], which depicts people performing diverse activities and annotates their 2D body joints. The model predicts joint heatmaps; its learned spatial gates provide input-dependent masks at gated blocks.

Table 1 summarizes their models and evaluation scales. We use official weights for all four models [18, 25, 34]. For each workload, we fix these weights, inputs, and upstream mask policy; the sparse paths receive the same resulting layer masks and differ only in their masked-convolution backend.

Table 1. Evaluation workloads.

	FireFlowNet [25]	**YOLOv8n/m** [18]	**DynConv Pose** [34]
Task	Optical flow	Detection	Human pose
Mask source	Ev-Conv [7]	DeltaCNN [27]	DynConv [34]
Dataset	MVSEC [42]	MOT16 [23]	MPII [1]
Input size	256×256	640×640	256×256
Parameters	0.056M	3.16M / 25.9M	6.89M
Eval. samples	196.8K	2,575	2,958

6.1.3 Execution Baselines. We compare WISEConv against three FP32 execution paths established by prior work.

- **Dense** uses the native PyTorch/cuDNN backend [5] with sparse execution disabled.
- **Tile skipping** follows the tile-sparse execution established by SBNet, Ev-Conv, and DeltaCNN [7, 27, 29].
- **Gather-scatter** uses the original DynConv CUDA backend for human pose [34] and an implementation equivalent to MMCV `MaskedConv2d` for FireFlowNet and YOLO [4].

6.1.4 Metrics and Methodology. Our primary metric is full-model GPU latency. The timed region includes mask propagation, worklist construction, all network operators, and output update, but excludes host-to-device input transfer, calibration, autotuning, and the one-time dense state initialization required by incremental models. We use CUDA events on the model's compute stream and process complete held-out sequences rather than isolated synthetic frames. Each sequence is repeated three times; we retain per-frame samples and report both aggregate latency and dispersion.

To explain latency, we report the required-work ratio ρ_{work}, useful-compute ratio η, convolution throughput, and the construction fraction of operator latency. Throughput here isolates the compute kernel; construction is reported separately as a fraction of operator latency, so readers can reconstruct the inclusive T of §2.2. We aggregate η as total active FLOPs over total scheduled FLOPs across all convolution layers, so each layer contributes in proportion to the work it actually schedules. Because layers differ in cost, an unweighted average of layer activities misstates the work a

mask removes, so we weight each layer by its dense MAC count,

$$\rho_{\text{work}} = \frac{\sum_l \alpha^l W^l_{\text{dense}}}{\sum_l W^l_{\text{dense}}}, \qquad \alpha^l = \frac{\|M^l\|_0}{H^l_{out} W^l_{out}}, \tag{11}$$

where W^l_{dense} is the dense MAC count of layer l. Task quality is measured with Average Endpoint Error (AEE) for optical flow, detection AP against the fixed evaluation references for YOLO, and PCKh for human pose. Accuracy comparisons use the same masks and operating point as latency measurements; they evaluate the upstream policy, while operator correctness is checked separately against dense convolution at every active output.

6.2 Full-Model Latency

6.3 Efficiency Analysis

6.4 Construction Overhead

6.5 Task Accuracy

References

[1] Mykhaylo Andriluka, Leonid Pishchulin, Peter Gehler, and Bernt Schiele. 2014. 2D Human Pose Estimation: New Benchmark and State of the Art Analysis. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3686–3693.

[2] Rik J. Bouwmeester, Federico Paredes-Vallés, and Guido C. H. E. de Croon. 2023. NanoFlowNet: Real-time Dense Optical Flow on a Nano Quadcopter. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 1996–2003.

[3] Lukas Cavigelli, Philippe Degen, and Luca Benini. 2017. CBinfer: Change-Based Inference for Convolutional Neural Networks on Video Data. In *Proc. Int. Conf. Distrib. Smart Cameras (ICDSC)*. 1–8.

[4] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, Zheng Zhang, Dazhi Cheng, Chenchen Zhu, Tianheng Cheng, Qijie Zhao, Buyu Li, Xin Lu, Rui Zhu, Yue Wu, Jifeng Dai, Jingdong Wang, Jianping Shi, Wanli Ouyang, Chen Change Loy, and Dahua Lin. 2019. MMDetection: Open MMLab Detection Toolbox and Benchmark. *CoRR* abs/1906.07155 (2019).

[5] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. *CoRR* abs/1410.0759 (2014).

[6] Christopher B. Choy, JunYoung Gwak, and Silvio Savarese. 2019. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

[7] Sankeerth Durvasula, Yushi Guan, and Nandita Vijaykumar. 2023. Ev-Conv: Fast CNN Inference on Event Camera Inputs for High-Speed Robot Perception. *IEEE Robotics Autom. Lett.* 8, 6 (2023), 3174–3181.

[8] Davide Falanga, Philipp Foehn, Peng Lu, and Davide Scaramuzza. 2018. PAMPC: Perception-Aware Model Predictive Control for Quadrotors. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2018, Madrid, Spain, October 1-5, 2018*. IEEE, 1–8. doi:10.1109/IROS.2018.8593739

[9] Davide Falanga, Suseong Kim, and Davide Scaramuzza. 2019. How Fast Is Too Fast? The Role of Perception Latency in High-Speed Sense and Avoid. *IEEE Robotics Autom. Lett.* 4, 2 (2019), 1884–1891. doi:10.1109/LRA.2019.2898117

[10] Davide Falanga, Kevin Kleber, and Davide Scaramuzza. 2020. Dynamic obstacle avoidance for quadrotors with event cameras. *Sci. Robotics* 5, 40 (2020), 9712. doi:10.1126/SCIROBOTICS.AAZ9712

[11] Ruibo Fan, Wei Wang, and Xiaowen Chu. 2024. DTC-SpMM: Bridging the Gap in Accelerating General Sparse Matrix Multiplication with Tensor Cores. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 253–267. doi:10.1145/3620666.3651378

[12] Michael Figurnov, Maxwell D. Collins, Yukun Zhu, Li Zhang, Jonathan Huang, Dmitry P. Vetrov, and Ruslan Salakhutdinov. 2017. Spatially Adaptive Computation Time for Residual Networks. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 1790–1799.

[13] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

[14] Yizhao Gao, Baoheng Zhang, Xiaojuan Qi, and Hayden Kwok-Hay So. 2023. DPACS: Hardware Accelerated Dynamic Neural Network Pruning through Algorithm-Architecture Co-design. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*, Tor M. Aamodt, Natalie D. Enright Jerger, and Michael M. Swift (Eds.). ACM, 237–251. doi:10.1145/3575693.3575728

[15] Benjamin Graham, Martin Engelcke, and Laurens van der Maaten. 2018. 3D Semantic Segmentation with Submanifold Sparse Convolutional Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[16] Amirhossein Habibian, Davide Abati, Taco S. Cohen, and Babak Ehteshami Bejnordi. 2021. Skip-Convolutions for Efficient Video Processing. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2695–2704.

[17] Drew Hanover, Antonio Loquercio, Leonard Bauersfeld, Angel Romero, Robert Penicka, Yunlong Song, Giovanni Cioffi, Elia Kaufmann, and Davide Scaramuzza. 2024. Autonomous Drone Racing: A Survey. *IEEE Trans. Robotics* 40 (2024), 3044–3067. doi:10.1109/TRO.2024.3400838

[18] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. 2023. Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics

[19] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman P. Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc. ACM Program. Lang.* 1, OOPSLA (2017).

[20] Fanrong Li, Gang Li, Xiangyu He, and Jian Cheng. 2021. Dynamic Dual Gating Neural Networks. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*. 5310–5319.

[21] Renyuan Liu, Yuyang Leng, Shilei Tian, Shaohan Hu, Chun-Fu (Richard) Chen, and Shuochao Yao. 2024. DynaSpa: Exploiting Spatial Sparsity for Efficient Dynamic DNN Inference on Devices. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems, SenSys 2024, Hangzhou, China, November 4-7, 2024*. ACM, 422–435. doi:10.1145/3666025.3699348

[22] Nico Messikommer, Daniel Gehrig, Antonio Loquercio, and Davide Scaramuzza. 2020. Event-Based Asynchronous Sparse Convolutional Networks. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 415–431.

[23] Anton Milan, Laura Leal-Taixé, Ian D. Reid, Stefan Roth, and Konrad Schindler. 2016. MOT16: A Benchmark for Multi-Object Tracking. *CoRR* abs/1603.00831 (2016).

[24] Asit K. Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. 2021. Accelerating Sparse Deep Neural Networks. *CoRR* abs/2104.08378 (2021).

[25] Federico Paredes-Vallés and Guido C. H. E. de Croon. 2021. Back to Event Basics: Self-Supervised Learning of Image Reconstruction for Event Cameras via Photometric Constancy. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3445–3454.

[26] Mathias Parger, Chengcheng Tang, Thomas Neff, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2023. MotionDeltaCNN: Sparse CNN Inference of Frame Differences in Moving Camera Videos with Spherical Buffers and Padded Convolutions. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*. 17246–17255.

[27] Mathias Parger, Chengcheng Tang, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2022. DeltaCNN: End-to-End CNN Inference of Sparse Frame Differences in Videos. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 12487–12496.

[28] Hongwu Peng, Xi Xie, Kaustubh Shivdikar, Md Amit Hasan, Jiahui Zhao, Shaoyi Huang, Omer Khan, David R. Kaeli, and Caiwen Ding. 2024. MaxK-GNN: Extremely Fast GPU Kernel Design for Accelerating Graph Neural Networks Training. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 683–698. doi:10.1145/3620665.3640426

[29] Mengye Ren, Andrei Pokrovsky, Bin Yang, and Raquel Urtasun. 2018. SBNet: Sparse Blocks Network for Fast Inference. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 8711–8720.

[30] Spconv Contributors. 2022. Spconv: Spatially Sparse Convolution Library. https://github.com/traveller59/spconv.

[31] Martin Sundermeyer, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. 2021. Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 13438–13444.

[32] Haotian Tang, Zhijian Liu, Xiuyu Li, Yujun Lin, and Song Han. 2022. TorchSparse: Efficient Point Cloud Inference Engine. In *Proceedings of Machine Learning and Systems (MLSys)*.

[33] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. TorchSparse++: Efficient Training and Inference Framework for Sparse Convolution on GPUs. In *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*. 225–239.

[34] Thomas Verelst and Tinne Tuytelaars. 2020. Dynamic Convolutions: Exploiting Spatial Sparsity for Faster Inference. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2320–2329.

[35] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martin Martin, Cewu Lu, Li Fei-Fei, and Silvio Savarese. 2019. DenseFusion: 6D Object Pose Estimation by Iterative Dense Fusion. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3343–3352.

[36] Kunyun Wang, Shuo Yang, Jieru Zhao, Wenchao Ding, Quan Chen, Jingwen Leng, and Minyi Guo. 2025. SparseTem: Boosting the Efficiency of CNN-Based Video Encoders by Exploiting Temporal Continuity. In *Advanced Parallel Processing Technologies - 16th International Symposium, APPT 2025, Athens, Greece, July 13-16, 2025, Proceedings*. 246–256.

[37] Diana Wofk, Fangchang Ma, Tien-Ju Yang, Sertac Karaman, and Vivienne Sze. 2019. FastDepth: Fast Monocular Depth Estimation on Embedded Systems. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 6101–6108.

[38] Zhenda Xie, Zheng Zhang, Xizhou Zhu, Gao Huang, and Stephen Lin. 2020. Spatially Adaptive Inference with Stochastic Feature Sampling and Interpolation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 531–548.

[39] Jiacheng Yang, Christina Giannoula, Jun Wu, Mostafa Elhoushi, James Gleeson, and Gennady Pekhimenko. 2024. Minuet: Accelerating 3D Sparse Convolutions on GPUs. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*. 786–802.

[40] Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. 2023. SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[41] Changqian Yu, Jingbo Wang, Chao Peng, Changxin Gao, Gang Yu, and Nong Sang. 2018. BiSeNet: Bilateral Segmentation Network for Real-Time Semantic Segmentation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 334–349.

[42] Alex Zihao Zhu, Dinesh Thakur, Tolga Özaslan, Bernd Pfrommer, Vijay Kumar, and Kostas Daniilidis. 2018. The Multi Vehicle Stereo Event Camera Dataset: An Event Camera Dataset for 3D Perception. *CoRR* abs/1801.10202 (2018).